

UD 8. UTILIZACIÓN AVANZADA DE CLASES

**Programación – 2º DAM
Curso 2025/2026**

Herencia

Herencia

- Constructores y Herencia
 - Uso de *super*
 - Otro uso de *super*
- Modificadores de acceso
- Redefinición de métodos heredados. Métodos *override*
- Clase *Object*
 - Método *toString()*
 - Método *equals()*

Herencia

- Polimorfismo
 - Tipos de polimorfismo
- Operador instanceof
- La palabra reservada final
- Abstracción
 - Abstracción. Métodos abstractos
- Interfaces

Herencia

La herencia es uno de los tres pilares básicos de la programación orientada a objetos.

Permite la creación de clasificaciones jerárquicas.

La herencia es la base de la reutilización del código.

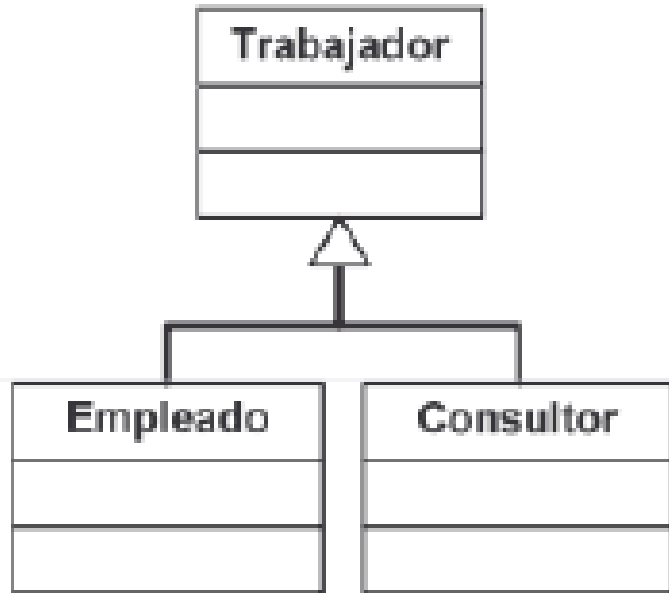
Cuando una clase deriva de una clase padre, ésta hereda todos los miembros y métodos de su antecesor.

También es posible redefinir (**override**) los miembros para adaptarlos a la nueva clase o bien ampliarlos.

En general, todas las subclases no solo adoptan las variables y comportamiento de las superclases, sino que los amplían.

Herencia

Un ejemplo de herencia sería:



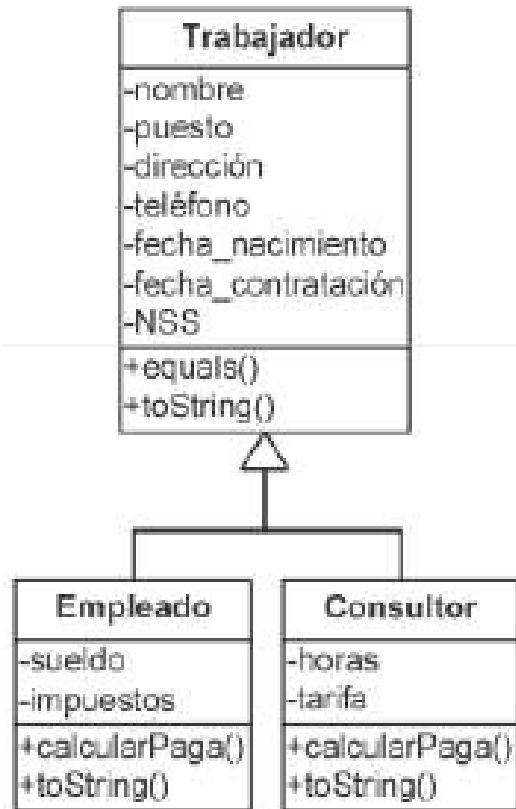
Trabajador es una clase genérica que sirve para almacenar datos como el nombre, la dirección, el número de teléfono o el número de la seguridad social de un trabajador.

Empleado es una clase especializada para representar los empleados que tienen una nómina mensual (encapsula datos como su salario anual o las retenciones del IRPF).

Consultor es una clase especializada para representar a aquellos trabajadores que cobran por horas (por ejemplo, registra el número de horas que ha trabajado un consultor y su tarifa horaria).

Herencia

Las clases **Empleado** y **Consultor** además de los atributos y de las operaciones que definen, heredan de **Trabajador** todos sus atributos y operaciones.



Herencia

unEmpleadoConcreto
-nombre -puesto -dirección -teléfono -fecha_nacimiento -fecha_contratación -NSS -sueldo -impuestos
+equals() +toString() +calcularPaga()

Un empleado concreto tendrá, además de sus atributos y operaciones como **Empleado**, todos los atributos correspondientes a la superclase **Trabajador**.

Herencia

Para indicar que una clase hereda de otra se etiqueta con la cláusula **extends** detrás del nombre de la clase.

```
import java.util.Date;

public class Trabajador {
    private String nombre, puesto, direccion, telefono, NSS;
    private Date fecha_nacimiento, fecha_contrato;

    //Constructor
    public Trabajador(String nombre, String NSS) {
        this.nombre=nombre;
        this.NSS=NSS;
    }

    //Comparación de objetos
    public boolean equals(Trabajador t) {
        return this.NSS.equals(t.NSS);
    }

    //Conversión en una cadena de caracteres
    public String toString() {
        return nombre+ " (NSS "+NSS+")";
    }
}
```


Herencia

Con la palabra reservada **extends** indicamos que Empleado es una subclase de Trabajador.

Con la palabra reservada **super** accedemos a miembros de la superclase desde la subclase.

Generalmente, en un constructor, lo primero que nos encontramos es una llamada al constructor de la clase padre con **super (...)**. Si no ponemos nada, se llama al constructor por defecto de la superclase antes de ejecutar el constructor de la subclase.

```
import java.util.Date;

public class Trabajador {
    private String nombre, puesto, direccion, telefono, NSS;
    private Date fecha_nacimiento, fecha_contrato;

    //Constructor
    public Trabajador(String nombre, String NSS) {
        this.nombre=nombre;
        this.NSS=NSS;
    }

    //Comparación de objetos
    public boolean equals(Trabajador t) {
        return this.NSS.equals(t.NSS);
    }

    //Conversión en una cadena de caracteres
    public String toString() {
        return nombre+ " (NSS "+NSS+")";
    }
}
```

```
public class Empleado extends Trabajador{
    private double sueldo;
    private double impuestos;
    private final int PAGAS=14;

    //Constructor
    public Empleado(String nombre, String NSS, double sueldo) {
        super(nombre, NSS);
        this.sueldo=sueldo;
        this.impuestos=0.3*sueldo;
    }

    //Nómina
    public double calcularPaga() {
        return (sueldo-impuestos)/PAGAS;
    }

    //toString
    public String toString() {
        return "Empleado "+super.toString();
    }
}
```

Herencia

La clase **Consultor** también define un método llamado **calcularPaga()**, si bien en este caso el cálculo se hace de una forma diferente por tratarse de un trabajador de un tipo distinto.

```
public class Constructor extends Trabajador{
    private int horas;
    private double tarifa;

    //Constructor
    public Constructor(String nombre, String NSS, int horas, double tarifa) {
        super(nombre, NSS);
        this.horas=horas;
        this.tarifa=tarifa;
    }

    //Paga por horas
    public double calcularPaga() {
        return horas*tarifa;
    }

    //toString
    public String toString() {
        return "Consultor "+super.toString();
    }
}
```

Herencia

Utilización de las clases:

```
public class GestionEmpleados {  
  
    public static void main(String[] args) {  
        //Declaración de variables;  
        Trabajador trabajador;  
        Empleado empleado;  
        Consultor consultor;  
  
        //Creación de objetos  
        trabajador=new Trabajador("Juan", "456");  
        empleado=new Empleado("Jose", "123", 24000.0);  
        consultor=new Consultor("Juan", "456", 10, 50.0);  
  
        //Salida estándar con toString()  
        System.out.println(trabajador);  
        System.out.println(empleado);  
        System.out.println(consultor);  
    }  
}
```

Juan (NSS 456)

Empleado Jose (NSS 123)

Consultor Juan (NSS 456)

Constructores y Herencia

En una jerarquía, es posible que tanto la superclase como las subclases tengan sus propios constructores.

El constructor de la superclase construye la porción de la superclase del objeto y el constructor de la subclase construye la parte de la subclase.

Constructores y Herencia

Ejemplo 1. SUPERCLASE SIN CONSTRUCTOR

```
//SUPERCLASE SIN CONSTRUCTOR
public class Trabajador {
    private String nombre, puesto, direccion, telefono, NSS;
    private Date fecha_nacimiento, fecha_contrato;

    //Comparación de objetos
    public boolean equals(Trabajador t) {
        return this.NSS.equals(t.NSS);
    }

    //Getters y Setters
    public String getNombre() {..}
    public void setNombre(String nombre) {..}
    public String getPuesto() {..}
    public void setPuesto(String puesto) {..}
    public String getDireccion() {..}
    public void setDireccion(String direccion) {..}
    public String getTelefono() {..}
    public void setTelefono(String telefono) {..}
    public String getNSS() {..}
    public void setNSS(String NSS) {..}
    public Date getFecha_nacimiento() {..}
    public void setFecha_nacimiento(Date fecha_nacimiento) {..}
    public Date getFecha_contrato() {..}
    public void setFecha_contrato(Date fecha_contrato) {..}

    //Conversión en una cadena de caracteres
    public String toString() {
        return nombre+ " (NSS "+NSS+")";
    }
}
```

Constructores y Herencia

Ejemplo 1. SUPERCLASE SIN CONSTRUCTOR

```
public class Empleado extends Trabajador{
    private double sueldo;
    private double impuestos;
    private final int PAGAS=14;

    //Constructor
    public Empleado(String nombre, String NSS, double sueldo) {
        this.setNombre(nombre);
        this.setNSS(NSS);
        this.sueldo=sueldo;
        this.impuestos=0.3*sueldo;
    }

    //Nómina
    public double calcularPaga() {
        return (sueldo-impuestos)/PAGAS;
    }

    //toString
    public String toString() {
        return "Empleado "+super.toString();
    }
}
```

Uso de *super*

Una subclase puede llamar al constructor definido en la superclase usando la palabra reservada `super`, cuya sintaxis es:

`super(parameter-list);`

`parameter-list` especifica los parámetros necesarios por el constructor de la superclase.

`super()` debe ser siempre la primera sentencia ejecutada dentro del constructor de la subclase.

La sobrecarga de constructores también puede darse en herencia

Uso de *super*

```
import java.util.Date;

public class Trabajador {
    private String nombre, puesto, direccion, telefono, NSS;
    private Date fecha_nacimiento, fecha_contrato;

    //Constructor
    public Trabajador(String nombre, String NSS) {
        this.nombre=nombre;
        this.NSS=NSS;
    }

    //Comparación de objetos
    public boolean equals(Trabajador t) {
        return this.NSS.equals(t.NSS);
    }

    //Conversión en una cadena de caracteres
    public String toString() {
        return nombre+ " (NSS "+NSS+")";
    }
}
```

```
public class Empleado extends Trabajador{
    private double sueldo;
    private double impuestos;
    private final int PAGAS=14;

    //Constructor
    public Empleado(String nombre, String NSS, double sueldo) {
        super(nombre, NSS);
        this.sueldo=sueldo;
        this.impuestos=0.3*sueldo;
    }

    //Nómina
    public double calcularPaga() {
        return (sueldo-impuestos)/PAGAS;
    }

    //toString
    public String toString() {
        return "Empleado "+super.toString();
    }
}
```


Otro uso de *super*

Hay un segundo uso de `super` que actúa similar a `this` excepto que siempre se refiere a la superclase de la subclase en la que es usado. Es decir, cuando en una subclase se accede a un atributo o método con la palabra `super` (`super.atributo` o `super.método`), nos estamos refiriendo a un atributo o método de su clase padre.

La sintaxis es:

`super.member`

donde *member* puede ser un método o un atributo.

Otro uso de *super*

```
class A {  
    int i;  
}  
  
class B extends A {  
    int i;  
  
    B(int a, int b) {  
        super.i = a;  
        i = b;  
    }  
  
    void show() {  
        System.out.println("i in superclass: " + super.i);  
        System.out.println("i in subclass: " + i);  
    }  
}  
  
class UseSuper {  
  
    public static void main(String args[]) {  
        B subOb = new B(1, 2);  
        subOb.show();  
    }  
}
```

Modificadores de acceso

Como ya sabemos, siguiendo la técnica de encapsulamiento, es común declarar un atributo de una clase como **private** para evitar usos no autorizados.

La herencia no anula la restricción de acceso privado.

Por lo tanto, aunque una subclase incluya todos los miembros de su superclase, no puede acceder directamente a estos miembros de la superclase que han sido declarados private, sino que tendrá que hacerlo a través de sus métodos get y set.

Modificadores de acceso

Además de los modificadores vistos (`private` y `public`), también nos encontramos con **`protected`**.

Este modificador nos permitirá que un miembro sea visible desde una subclase, consiguiendo así una mayor flexibilidad y facilitando por tanto la herencia.

En resumen, un miembro **`protected`** es visible en las clases vecinas (mismo paquete), no es visible en las clases externas, pero siempre es visible, independientemente del paquete al que pertenezca, desde una clase hija.

Modificadores de acceso

	Visible desde...			
	la propia clase	clases vecinas	subclases	clases externas
<code>private</code>	✓			
<i>sin modificador</i>	✓	✓		
<code>protected</code>	✓	✓	✓	
<code>public</code>	✓	✓	✓	✓

Redefinición de métodos heredados.

Métodos *override*

En una jerarquía de clases, cuando un método en una subclase tiene el **mismo encabezado y el mismo valor de retorno** que un método de su superclase, se dice que el método de la subclase **sobreescribe** (***override***) el método de la superclase.

Cuando un método **sobreescrito** es llamado desde una subclase, siempre estará referenciando al método de la subclase y el método definido en la superclase será obviado.

Aunque es opcional, los métodos sustituidos en las subclases se suelen marcar con la anotación `@Override`, que indica que el método es una sustitución y overriding de un método de la superclase.

Redefinición de métodos heredados.

Métodos *override*

Por tanto, para hacer overriding de un método de la superclase, es imprescindible que el que lo sustituye en la subclase tenga el mismo nombre, la misma lista de parámetros de entrada y el mismo tipo devuelto. En caso contrario, se producirá un error de compilación.

Redefinición de métodos heredados.

Métodos *override*

```
public class Persona {
    private String nombre;
    private int edad;
    private double estatura;

    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public int getEdad() {
        return edad;
    }
    public void setEdad(int edad) {
        this.edad = edad;
    }
    public double getEstatura() {
        return estatura;
    }
    public void setEstatura(double estatura) {
        this.estatura = estatura;
    }
    public void mostrarDatos() {
        System.out.println(nombre);
        System.out.println(edad);
        System.out.println(estatura);
    }
}
```

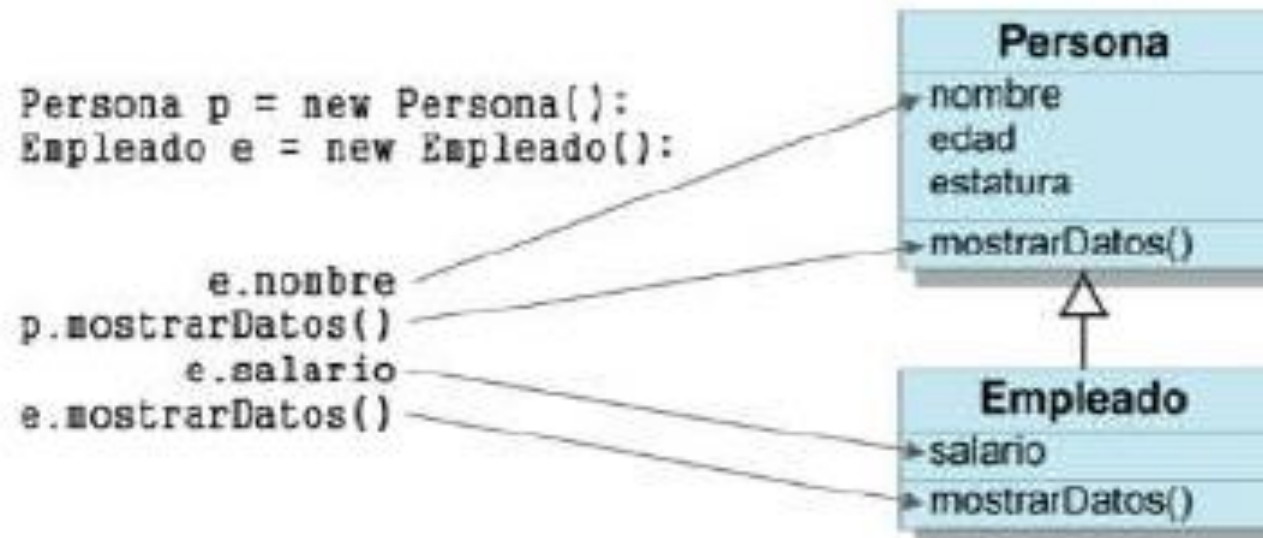
```
public class Empleado extends Persona{
    double salario;

    @Override //significa que sustituye un método de la superclase
    public void mostrarDatos() {
        System.out.println(super.getNombre());
        System.out.println(super.getEdad());
        System.out.println(super.getEstatura());
        System.out.println(this.salario);
    }
}
```


Redefinición de métodos heredados.

Métodos *override*

El método **mostrarDatos()** definido en **Empleado** sustituye al método, con el mismo nombre y los mismos parámetros, de **Persona**. Si la lista de parámetros no es la misma, no hay overriding, si no que estaríamos haciendo una **sobrecarga** del método **mostrarDatos()**.



Clase *Object*

La clase **Object** del paquete java.Lang es una clase especial de la que heredan, directa o indirectamente, todas las clases de Java.

Todas las clases que componen la API de Java descienden de esta clase Object. Incluso cualquier clase que implementemos nosotros, por defecto, hereda a su vez de Object.

Por ejemplo, la definición de la clase Persona:

```
class Persona{    ...    }
```

es en realidad, equivalente a:

```
class Persona extends Object{    ...    }
```

Y cualquier clase que herede de Perrsona está heredando, a su vez, de Object.

Clase *Object*

El objetivo de que todas las clases hereden de Object es:

- Que todas las clases implementen un conjunto de métodos que son de uso universal en Java, como realizar comparaciones entre objetos, clonarlos o representar un objeto como cadena. La función de estos métodos es ser reimplementados a la medida de cada clase.
- Poder referenciar cualquier objeto, de cualquier tipo, mediante una variable de tipo Object.

Método *toString()*

Este método está pensado para que devuelva una cadena que represente al objeto que lo invoca con toda la información que interese mostrar.

Tiene el prototipo:

```
public String toString()
```

Su implementación en la clase `Object` consiste en devolver el nombre cualificado de la clase a la que pertenece el objeto, seguida de una arroba (@) junto a la referencia del objeto.

Esta implementación por defecto no es útil para representar la mayoría de los objetos, por lo que nos vemos obligados a realizar un overriding de `toString()` en cada clase, que es donde se encuentra la información que queremos representar.

Método *toString()*

La sobreescritura del método `toString()` obliga a que sea declarado public, igual que en la clase `Object`, ya que **todo método que sustituye a otro debe tener, al menos, el mismo nivel de acceso.**

Por defecto, `System.out.println()` invoca por defecto el método `toString()`.

Ejemplo:

```
Persona p=new Persona("Claudia", 8, 1.20);  
System.out.println(p.toString());
```

//sería equivalente a

```
System.out.println(p);
```

Método *equals()*

Compara dos objetos y decide si son iguales, devolviendo true en caso afirmativo y false en caso contrario. Su prototipo en la clase Object es:

```
public boolean equals(Object otro)
```

El operador == es útil para comparar tipos primitivos, pero no sirve para comparar objetos, ya que en este caso compara sus referencias, sin fijarse en su contenido. Por ejemplo:

```
Persona a = new Persona("Claudia", 8, 1.20);  
Persona b = new Persona("Claudia", 8, 1.20);  
System.out.println(a==b);    //false
```

El resultado es false porque la comparación se hace atendiendo a las referencias de los objetos, que son distintas.

Método *equals()*

El prototipo de **equals()** tiene un parámetro de entrada de tipo Object para poder comparar objetos de cualquier clase. Este prototipo debe mantenerse al hacer **overriding** en cualquier clase, de lo contrario no sería overriding, sino sobrecarga. Pero, para acceder a los atributos del objeto pasado como parámetro, tenemos que informar al compilador de que, en realidad, es un objeto Persona. Esto se consigue por medio de un **cast**, como veremos en el siguiente ejemplo.

Método *equals()*

Vamos a reimplementar el método equals para comparar objetos de la clase Persona, considerando que dos personas son iguales si tienen mismo nombre y edad.

```
@Override
public boolean equals(Object otro) {    //compara this con otro
    Persona otraPersona= (Persona) otro;
    boolean iguales;
    if(this.nombre.equals(otraPersona.nombre) &&
this.edad==otraPersona.edad) {
        iguales=true;
    }else {
        iguales=false;
    }
    return iguales;
}
```


Método *equals()*

En la segunda línea del ejemplo anterior se está haciendo un cast a Persona.

En Java, **cast** (o **casteo**) es la conversión de un tipo de dato a otro. Se usa cuando se necesita tratar un valor de un tipo como si fuera de otro tipo.

En el caso del método *equals()*, siempre es obligatorio ese cast porque su prototipo tiene que ser el mismo que en la clase ***Object***, donde el parámetro de entrada es de tipo Object.

Pero para poder acceder a los atributos **nombre** y **edad** de la clase Persona necesitamos que la variable **otro** sea de tipo **Persona**.

Polimorfismo

El polimorfismo es un principio de la programación orientada a objetos (POO) que permite que una **misma interfaz o método** pueda tener **diferentes comportamientos** según el contexto.

En Java, el polimorfismo se da cuando **una clase hija puede sobrescribir métodos de su clase padre** o cuando una interfaz tiene múltiples implementaciones.

Al redefinir métodos, objetos de diferentes tipos pueden responder de forma diferente a la misma llamada (y podemos escribir código de forma general sin preocuparnos del método concreto que se ejecutará en cada momento).

Tipos de polimorfismo

1. Polimorfismo en tiempo de compilación (sobrecarga de métodos - Overloading)

- Se da cuando varios métodos tienen el mismo nombre, pero diferente número o tipo de parámetros.
- Se determina en tiempo de compilación.

Tipos de polimorfismo

Ejemplo: Sobrecarga de métodos

```
class Calculadora {  
    int sumar(int a, int b) {  
        return a + b;  
    }  
    // Mismo método, distinto tipo de parámetros  
    double sumar(double a, double b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculadora calc = new Calculadora();  
        // Llama al método con enteros  
        System.out.println(calc.sumar(5, 3));  
        // Llama al método con doubles  
        System.out.println(calc.sumar(2.5, 3.2));  
    }  
}
```

Tipos de polimorfismo

2. Polimorfismo en tiempo de ejecución (sobreescritura de métodos - Overriding)

- Se da cuando una subclase redefine un método de una superclase con la misma firma.
- Se determina en tiempo de ejecución dependiendo del tipo real del objeto.

Tipos de polimorfismo

Ejemplo: Sobreescritura de métodos

```
class Animal {  
    void hacerSonido() {  
        System.out.println("Sonido genérico de animal");  
    }  
}  
  
class Perro extends Animal {  
    @Override  
    void hacerSonido() { //Sobrescribe el método de la superclase  
        System.out.println("Guau guau");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal miAnimal = new Perro();  
        miAnimal.hacerSonido(); //Ejecuta el método de Perro  
    }  
}
```

Operador instanceof

El operador **instanceof** en Java es una herramienta clave para la verificación de tipos en tiempos de ejecución. Permite comprobar si un objeto es una instancia de una clase específica o si pertenece a una jerarquía de herencia determinada.

Su sintaxis es:

objeto **instanceof** *Clase*

Siendo:

objeto es la referencia que se desea evaluar

Clase es la clase o interfaz con la que se compara

Este operador devuelve **true** si es una instancia de la clase o una de las subclases; en caso contrario, devuelve **false**.

Operador instanceof

Suponemos que tenemos una clase **Animal**, y una clase **Perro** que hereda de **Animal**:

```
public class Animal{  
  
public class Perro extends Animal{  
  
public class Main{  
    public static void main (String[] args){  
        Animal miAnimal=new Perro();  
        System.out.println(miAnimal instanceof Perro);//true  
        System.out.println(miAnimal instanceof Animal);//true  
    }  
}
```

} //Aquí, **miAnimal** es una instancia de **Perro**, pero también se considera de tipo **Animal**, por lo que ambas evaluaciones devuelven **true**.

La palabra reservada final

En Java, usando la palabra reservada **final**, podemos:

1. **Definir constantes simbólicas.**
2. **Evitar que un método se pueda redefinir en una subclase.**
3. **Evitar que puedan crearse subclases de una clase dada.**

La palabra reservada final

2. Evitar que un método se pueda redefinir en una subclase.

```
public class Consultor extends Trabajador{  
    ...  
    public final double calcularPaga() {  
        return horas*tarifa;  
    }  
    ...  
}
```

Aunque creemos subclases de Consultor, el dinero que se le pague siempre será en función de las horas que trabaje y de su tarifa horaria (y eso no se puede cambiar).

La palabra reservada final

3. Evitar que puedan crearse subclases de una clase dada.

```
public final class Circulo extends Figura  
...  
public final class Cuadrado extends Figura  
...
```

Al usar final, tanto Circulo como Cuadrado son ahora clases de las que no se pueden crear subclases.

Abstracción

Una **clase abstracta** es aquella clase de la que no se tiene intención de crear objetos, sino que únicamente sirve para unificar datos u operaciones de subclases.

- Supongamos un esquema de herencia que consta de la clase **Profesor** de la que heredan **ProfesorInterino** y **ProfesorTitular**.
- Es posible que todo profesor sea o bien ProfesorInterino o bien ProfesorTitular, es decir, que no vayan a existir instancias de la clase Profesor.
- Entonces, ¿qué sentido tiene crear la clase Profesor?

Abstracción

El sentido está en que una superclase permite unificar campos y métodos de las subclases, evitando la repetición de código y unificando procesos.

Sin embargo, si no se tiene intención de crear objetos la declararemos en Java de forma especial: como **clase abstracta**.

La declaración de que una clase es abstracta se hace con la sintaxis:

```
public abstract class NombreDeLaClase { ... }
```

Ejemplo:

```
public abstract class Profesor
```

Abstracción

Sin embargo, sigue funcionando como superclase de forma similar a como lo haría una superclase “normal”. La diferencia principal radica en que **no se pueden crear objetos de esta clase.**

Si una clase no se declara usando abstract se cataloga como “clase concreta”.

Una clase abstracta para Java es una clase de la que nunca se van a crear instancias: simplemente va a servir como superclase a otras clases.

No se puede usar la palabra clave new aplicada a clases abstractas.

Abstracción. Métodos abstractos

A su vez, **las clases abstractas suelen contener métodos abstractos.**

Hay métodos que no podemos implementar en una clase determinada por falta de datos, pero sí en sus subclases, donde se han añadido los atributos necesarios. La idea es **implementarlos vacíos, solo con el prototipo, en la superclase, y hacer overriding en las subclases**, donde ya disponemos de la información necesaria para implementar los detalles.

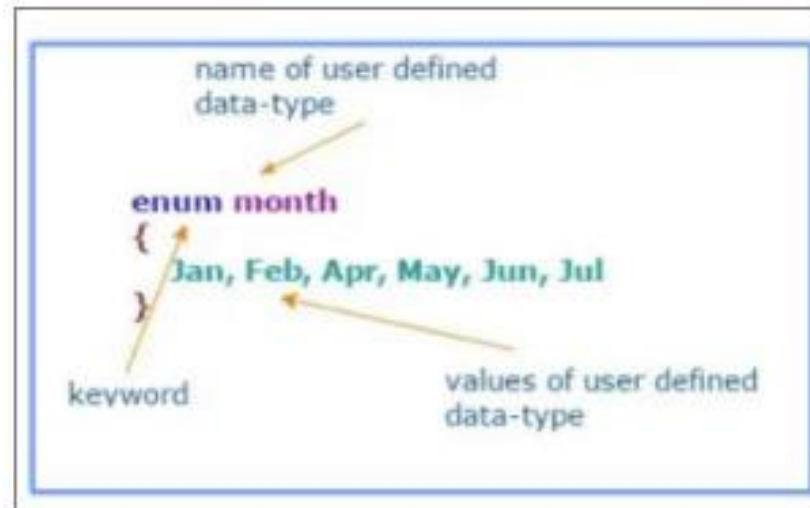
Un método definido en una clase, pero cuya implementación se delega en las subclases, se conoce como ***abstracto***. Para declarar un método abstracto se le antepone el modificador **abstract** y se declara el prototipo, sin escribir el cuerpo de la función. **Ejemplo:**

```
abstract void mostrarDatos();
```

Enumeraciones

Una enumeración (enum) es una clase "especial" en Java que representa un conjunto fijo de valores constantes. Se usa cuando una variable solo puede tomar uno de un conjunto limitado de valores predefinidos.

No se pueden crear objetos de una clase enum, ya que su propósito es definir un conjunto fijo de valores. Sin embargo, se pueden usar dentro de otras clases para representar estados o categorías. Si un enum tiene un constructor, este debe ser privado para evitar que se creen nuevas instancias fuera de los valores predefinidos.



Enumeraciones

Ejemplo: si pensamos en un jugador de baloncesto, éste puede ser de distinto tipo en función de la posición en la que juegue: BASE, ESCOLTA, ALERO, ALA_PIVOT, PIVOT. Por tanto, podemos decir que sólo serán válidos estos tipos de jugadores, es decir, las posiciones en las que puede jugar un jugador de baloncesto son finitas y se pueden enumerar: BASE, ESCOLTA, ALERO, ALA_PIVOT, PIVOT.

Enumeraciones

Sintaxis

```
public enum nombre_enumeracion {  
    VALOR_1, VALOR_2, ..., VALOR_N  
}
```

//Por convenio, los nombre de los enumerados se crean en MAYÚSCULAS

En nuestro ejemplo podemos crearnos una clase Enum llamada “Posicion”:

```
public enum Posicion{  
    BASE, ESCOLTA, ALERO, ALA_PIVOT, PIVOT  
}
```

Y si ahora queremos usar uno de sus valores en otra clase, por ejemplo, ALERO, haremos: posicion.ALERO

Enumeraciones

Tenemos diferentes formas de acceder a los valores de un enum:

- Directamente asignar un valor:

Abstracción. Métodos abstractos

Las subclases deberán implementar el método **mostrarDatos()**, cada una con las particularidades específicas de la clase, que no se conocen al nivel de la superclase.

Toda clase que tiene un método abstracto debe ser declarada, a su vez, *abstract*.

Todas las clases que hereden de una clase abstracta tiene dos opciones:

- Implementar todos los métodos abstractos de la clase padre.
- Dejar alguno de los métodos abstractos de la superclase sin implementar, y en ese caso, la subclase también será abstracta.

Una clase abstracta puede tener método concreto (implementado) y algunos atributos definidos, que heredarán las subclases, pudiendo hacer sustitución y ocultación de ellos.

Abstracción. Métodos abstractos

Un método abstracto tiene estas peculiaridades:

- No tiene cuerpo (llaves), termina con punto y coma.
- Un método abstracto nunca va a ser ejecutado (porque no tiene cuerpo).
- Solo puede existir dentro de una clase abstracta. Es decir, si una clase incluye un método abstracto, forzosamente la clase será abstracta.
- Si una subclase no implementa un método abstracto de la superclase → fuerza a ser una **subclase abstracta**.
- Para que la subclase sea concreta → debe implementar métodos sobrescritos para todos los métodos abstractos de sus superclases.

Abstracción. Métodos abstractos

Resumen reglas métodos abstractos:

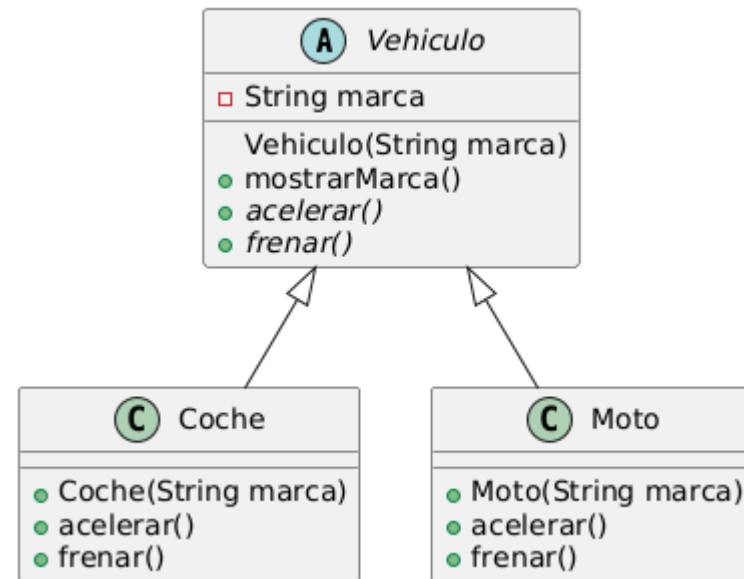
- Solo pueden declararse en una clase abstracta.
- No tiene cuerpo (solo la firma del método)
- Las subclases deben sobrecribirlo obligatoriamente (a menos que también sean abstractas).

Su sintaxis es:

```
abstract tipoRetorno nombreMetodo (parametros) ;
```

Abstracción. Métodos abstractos

Ejemplo. Imagina que estamos desarrollando un sistema para diferentes tipos de vehículos. Queremos que todos los vehículos puedan acelerar y frenar, pero cada uno lo hará de forma diferente. Un diagrama de clases para este ejemplo sería:



Abstracción. Métodos abstractos

```
//Clase abstracta que define el concepto de Vehículo
abstract class Vehiculo {
    protected String marca;
    // Constructor
    public Vehiculo(String marca) {
        this.marca = marca;
    }
    // Método abstracto (sin implementación)
    abstract void acelerar();
    // Método abstracto (sin implementación)
    abstract void frenar();
    // Método concreto (tiene implementación)
    public void mostrarMarca() {
        System.out.println("Marca del vehículo: " + marca);
    }
}
```


Abstracción. Métodos abstractos

```
// Clase concreta que extiende de Vehiculo
class Coche extends Vehiculo {
    public Coche(String marca) {
        super(marca);
    }
    @Override
    void acelerar() {
        System.out.println("El coche " + marca + " está acelerando.");
    }
    @Override
    void frenar() {
        System.out.println("El coche " + marca + " está frenando.");
    }
}
```

Abstracción. Métodos abstractos

```
// Otra subclase con diferente implementación
class Moto extends Vehiculo {
    public Moto(String marca) {
        super(marca);
    }
    @Override
    void acelerar() {
        System.out.println("La moto " + marca + " acelera rápido.");
    }
    @Override
    void frenar() {
        System.out.println("La moto " + marca + " frena con cuidado");
    }
}
```

Abstracción. Métodos abstractos

Uso de la abstracción en main. Podemos trabajar con la superclase Vehículo sin preocuparnos de detalles específicos

```
public class Main {  
    public static void main(String[] args) {  
        Vehiculo miCoche = new Coche("Toyota");  
        Vehiculo miMoto = new Moto("Yamaha");  
  
        miCoche.mostrarMarca();  
        miCoche.acelerar();  
        miCoche.frenar();  
  
        miMoto.mostrarMarca();  
        miMoto.acelerar();  
        miMoto.frenar();  
    }  
}
```

Interfaces

En Java, una **interfaz** es una estructura que define un conjunto de métodos que una clase debe implementar. Sirve para establecer un “**contrato**” entre las clases sin definir la implementación de los métodos.

Supongamos que vamos a trabajar con clases de animales. De ellos, unos tienen la capacidad de emitir un sonido; por ejemplo, los perros, los gatos o los lobos, y otros no. Los primeros, por tanto, tendrán el método:

```
void voz()
```

Sin embargo, la forma en que se ejecuta dicho método es distinta según la clase. En un objeto de la clase Gato, la ejecución de voz hará que aparezca en la pantalla la cadena “Miau”. En cambio, en un objeto Perro mostrará “Guau”. Pero todos ellos tienen en común que implementan el método voz().

Interfaces

Para gestionar la situación anterior, podríamos crear la clase abstracta `Animal`, de la que heredarían las clases `Gato` y `Perro`, con el método abstracto `voz()`, pero eso descartaría otras clases que no son animales, pero emiten sonido, como `Persona` o `Piano`.

Esta es la funcionalidad de las interfaces, expresar esa funcionalidad común de algunas clases que no tienen otra relación entre ellas, en este caso las que tienen implementado el método **`voz()`**.

En el ejemplo, hablaríamos de la interfaz `Sonido`, que consiste en implementar el método **`voz()`**. Diremos que las clases `Perro`, `Gato` y `Lobo` implementan la interfaz `Sonido`, ya que, entre sus métodos está la clase `voz()`, mientras que `Caracol` y `Lagartija`, no.

Interfaces

Una interface es una **clase abstracta** donde:

- todos sus métodos son abstractos
- todos sus atributos son constantes
- desde Java 8 puede además contener métodos por defecto y estáticos (implementados)
- desde Java 9 puede además contener métodos privados (implementados)

Interfaces

Métodos por defecto

- Se usa la palabra clave **default**.

Sirve para añadir funcionalidad a una interfaz sin obligar a modificar todas las clases que la implementan.

Ejemplo:

```
interface Vehiculo {  
  
    int getVelocidadMaxima();  
  
    default String mostrarDatos() {  
        return "Velocidad máxima: " + getVelocidadMaxima();  
    }  
}
```

Las clases que implementen la interfaz:

- pueden usar el método tal cual
- o sobreescribirlo

```
Coche c = new Coche();  
System.out.println(c.mostrarDatos());
```

Interfaces

Métodos estáticos

- Son métodos que **pertenecen a la interfaz**, no a las clases que la implementan.
- Se llaman usando **el nombre de la interfaz**.

Ejemplo:

```
interface Vehiculo {  
  
    static String tipoVehiculo() {  
        return "Medio de transporte";  
    }  
}
```

Se usaría así:

```
System.out.println(Vehiculo.tipoVehiculo());
```

Estos métodos no se heredan y no se pueden sobrescribir.

Interfaces

Métodos privados

Se usan **solo dentro de la interfaz**.

Sirven para **reutilizar código entre métodos default o static**.

No pueden ser usados por las clases que implementan la interfaz.

Ejemplo:

```
interface Vehiculo {  
  
    int getVelocidadMaxima();  
  
    default String mostrarDatos() {  
        return formatearDatos();  
    }  
  
    private String formatearDatos() {  
        return "Velocidad máxima: " + getVelocidadMaxima();  
    }  
}
```

El método formatearDatos() solo puede ser llamado **dentro de la interfaz**.

Estos métodos no se heredan y no se pueden sobrescribir.

Interfaces

Una interface se diferencia de una clase en que:

- no se puede instanciar (ya que es una clase abstracta)
- no contiene constructores
- si contiene atributos todos deben ser public static final (lo son, aunque no se indique)

Una clase puede heredar de varias interfaces, lo que significa que con las interfaces se permite la herencia múltiple.

La **interfaz** indica **qué hay que hacer** y la **clase** especifica **cómo se hace**.

Interfaces. Definición de una interfaz

La sintaxis de un interfaz es la siguiente:

```
interface Nombre_Interfaz{  
    métodos de la interfaz  
}
```

Por ejemplo, para Sonido sería:

```
interface Sonido{  
    void voz(); //método de la interfaz  
}
```

En la definición aparece el nombre del método, la lista de parámetros de entrada (si la tiene), y el tipo devuelto (en nuestro caso, void). No se implementa el cuerpo del método, ya que este depende de la clase concreta que implemente Sonido.

Interfaces. Implementación de una interfaz

Cuando una clase implementa una interfaz, la declara en la cabecera con la palabra reservada **implements** seguida del nombre de la interfaz.

```
class NombreClase implements NombreInterfaz{  
    ...  
}
```

La declaración de un método de una interfaz en una clase tiene que declararse **public**.

Es en la definición de cada clase donde se decide qué hace exactamente voz(). Por ejemplo, la clase Perro se definiría así:

```
class Perro implements Sonido{  
    public void voz(){  
        System.out.println("Guau");  
    }  
    ... //resto de la implementación de la clase Perro  
}
```

Interfaces. Implementación de una interfaz

En definitiva, cuando una clase declara en su encabezamiento que tiene implementada una interfaz concreta, se está comprometiendo a tener implementadas en su definición todas las funcionalidades de esa interfaz. En caso contrario, el compilador se encargará de mostrar un mensaje de error. Las interfaces, sin embargo, no son instanciables por sí mismas, solo podemos crear objetos de las clases que las implementan.

Una interfaz puede tener uno o varios métodos declarados, pero no implementados en la propia interfaz. Todos estos métodos, aunque normalmente no se indique, **son métodos abstractos**.

Interfaces. Atributos de una interfaz

Una interfaz también puede declarar atributos. Por ejemplo, si queremos llevar la cuenta de las sucesivas versiones de nuestra interfaz Sonido, podemos definir el atributo entero versión. La definición de la interfaz tendría la siguiente forma:

```
interface Sonido{  
    int versión=1;  
    void voz();  
}
```

El atributo versión se convertirá automáticamente en un atributo de las clases Gato y Perro, y solo podrá cambiarse en la definición de Sonido, pero no en las clases que la implementan, ya que los atributos definidos en una interfaz son static y final por defecto. Se accede a ellos, solo para lectura, a través del nombre de la interfaz o de una clase que la implemente, pero no desde una instancia. Por ejemplo:

```
System.out.println(Sonido.versión);
```

O bien:

```
System.out.println(Perro.versión);
```